

Tema: Requerimientos Ágiles e Historias de Usuario (User Stories)

Temas Abordados (Índice)

Tema 1: Filosofía y Fundamentos de los Requerimientos Ágiles

El enfoque ágil: balance metodológico y adaptabilidad.

El desperdicio del BRUF (Big Requirements Up Front) y el costo del desarrollo tradicional.

Principios del Manifiesto Ágil y Lean aplicados a la Ingeniería de Requerimientos.

Taxonomía de requerimientos: Negocio, Usuario y Software (Dominio del problema vs. Dominio de la solución).

Tema 2: La Esencia de las Historias de Usuario (User Stories)

Origen del concepto y naturaleza multipropósito.

El modelo de las 3C (Card, Conversation, Confirmation).

Formato estándar de expresión de Historias de Usuario.

Modelado de Roles de Usuario y la Tarjeta de Rol.

Tema 3: Atributos de Calidad y Criterios de Aceptación

El modelo INVEST para la validación de historias de usuario.

Criterios de aceptación: definición, límites y buenas prácticas (Intención vs. Solución).

Pruebas de Aceptación y el Desarrollo Dirigido por Pruebas de Aceptación (ATDD/STDD).

Gestión de detalles finos y documentación complementaria.

Antipatrones de redacción: Qué "huele mal" en una Historia de Usuario (Story Smells).

Tema 4: Estimación de Tamaño y Esfuerzo en Entornos Ágiles

El contraste fundamental entre Tamaño y Esfuerzo.

Concepto y definición de Story Point (Puntos de Historia).

Escalas de estimación ágil (Fibonacci).

Estimación colaborativa mediante Planning Poker: prerequisites, mecánica y algoritmo paso a paso.

Tema 5: Gestión y Estructuración Avanzada de Requerimientos

Jerarquía de abstracción: Temas (Themes), Épicas (Epics), Historias de Usuario y Granos de Arena.

Estrategia de descomposición: Porciones Verticales (Vertical Slices) vs. Porciones Horizontales y entrega de valor.

Spikes Técnicos y Funcionales: reducción del riesgo e incertidumbre.

El Refinamiento del Backlog (Grooming) y la priorización del Product Owner.

Criterios de entrada y salida: Definición de Listo (DoR) y Definición de Hecho (DoD).

Tema 1: Filosofía y Fundamentos de los Requerimientos Ágiles

La Ingeniería de Requerimientos convencional asume que es posible y deseable especificar completamente los requisitos de un sistema de software antes de iniciar su construcción. Sin embargo, en sistemas complejos, la **incapacidad de predecir** el comportamiento real de los usuarios y el dinamismo del mercado hacen que este enfoque prescriptivo resulte ineficiente y riesgoso. El agilismo propone un cambio de paradigma radical en el descubrimiento, modelado y gestión de las necesidades de software.

El Enfoque Ágil: Balance y Adaptabilidad

Los métodos ágiles no proponen la ausencia total de documentación ni la falta de rigor, sino que se definen como un compromiso útil entre nada de proceso y demasiado proceso. Su filosofía metodológica se asienta sobre dos pilares fundamentales respecto a la gestión tradicional:

Adaptabilidad sobre Predicción: Los métodos ágiles asumen el cambio de requerimientos como algo inevitable, estructurando un proceso capaz de reaccionar y adaptarse continuamente a las variaciones del negocio, en lugar de intentar predecir rígidamente mediante planes estáticos.

Orientación a la gente sobre el Proceso: Priorizan la colaboración humana, la autoorganización y el flujo de comunicación informal sobre el seguimiento dogmático de plantillas e interfaces procedimentales estrictas.

El Costo del BRUF (Big Requirements Up Front)

El enfoque tradicional requiere una etapa inicial masiva de especificación denominada BRUF (Big Requirements Up Front), orientada a firmar contractualmente un alcance cerrado. La evidencia empírica acumulada revela que este método genera un desperdicio sistemático en el desarrollo de software.

De acuerdo con el estudio clásico de Jim Johnson (The Standish Group, XP 2002 Keynote) sobre la utilización real de las características y funciones de software en sistemas de información típicos desarrollados bajo enfoques tradicionales, se evidencia lo siguiente:

- 45% de las funciones nunca son utilizadas por los usuarios finales.
- 19% de las funciones son utilizadas de manera rara o marginal.
- 16% de las funciones se usan solo de forma ocasional (algunas veces).
- Únicamente un 20% del software construido se utiliza de forma frecuente o constante (13% a menudo y 7% siempre).

El BRUF obliga a los analistas y clientes a diseñar y especificar detalladamente un 80% de funcionalidades que carecerán de valor de negocio práctico, consumiendo tiempo, presupuesto y esfuerzo que podrían destinarse a refinar las características medulares del producto. En **entornos ágiles**, la regla de oro consiste en **analizar cuando sea necesario**, no antes (***Just-In-Time Requirements***), difiriendo el análisis detallado de una funcionalidad hasta el momento previo a su implementación real para evitar inventarios innecesarios de requerimientos obsoletos.

Principios del Manifiesto Ágil y Lean Aplicados a Requerimientos

La gestión ágil de requerimientos se fundamenta en principios explícitos del Manifiesto Ágil y de la filosofía Lean Software Development:

Principios Ágiles Clave

Satisfacción del cliente mediante entregas tempranas y continuas: Entregar incrementos funcionales frecuentemente (cada dos semanas a un mes) para validar de manera empírica que se está construyendo el producto correcto.

Aceptar requisitos cambiantes: El cambio se percibe como una ventaja competitiva para el cliente, no como una disrupción al plan. El proceso debe permitir repriorizar o remover requerimientos de la pila de trabajo en cualquier momento.

Colaboración diaria: Técnicos (desarrolladores, testers) y no técnicos (Product Owner, usuarios, stakeholders) deben trabajar juntos diariamente durante todo el proyecto para resolver dudas semánticas y refinar la solución.

Comunicación cara a cara: El medio más eficiente y eficaz de transmitir información al equipo es la conversación directa. Permite que fluya no sólo la información vocal, sino también la subvocal, gestual y contextual, logrando ciclos de retroalimentación inmediatos frente a la rigidez de los documentos en prosa.

Requerimientos emergentes: Las mejores arquitecturas, diseños y requerimientos emergen de equipos autoorganizados a través de un proceso iterativo de descubrimiento.

Principios Lean Clave

Eliminar el desperdicio (Waste): Evitar la especificación prematura de características que nunca se utilizarán o que cambiarán antes de programarse.

Diferir compromisos (Decidir lo más tarde posible): Mantener las opciones abiertas y tomar decisiones de diseño y requerimientos en el último momento responsable (Last Responsible Moment), cuando se cuenta con la mayor cantidad de información y experiencia real.

Ver el todo (Optimize the Whole): Comprender el sistema desde una perspectiva sistémica, asegurándose de que cada historia aporte valor directo al flujo global del negocio del cliente, en lugar de optimizar requerimientos locales aislados.

Taxonomía de Requerimientos: Tradicional vs. Ágil

En la Ingeniería de Software tradicional se estructuran jerarquías estrictas desde el Dominio del Problema hasta el Dominio de la Solución:

Requerimientos de Negocio: Objetivos de alto nivel de la organización (ej. disminuir un X% el tiempo de procesamiento manual).

Requerimientos de Usuario: Tareas que el usuario final debe poder realizar utilizando el sistema (ej. realizar consultas en línea de estados de cuenta).

Requerimientos de Software (Funcionales y No Funcionales): Especificaciones técnicas detalladas del comportamiento del sistema y sus restricciones técnicas de plataforma, base de datos e implementación (ej. generar reportes de saldos en formato PDF, servidores en la nube).

En el desarrollo tradicional, el analista intenta capturar esto de forma prescriptiva, asumiendo una falsa estabilidad. Sin embargo, la realidad de los proyectos demuestra que "el usuario solo sabe lo que quiere cuando recibe lo que pidió".

El enfoque ágil reemplaza esta jerarquía documental rígida por una **pila de requerimientos priorizada por valor de negocio** (Product Backlog), gestionada activamente por el **Product Owner** (PO). Cada iteración o Sprint implementa los requerimientos de mayor prioridad de la pila, permitiendo reajustar, agregar o remover ítems dinámicamente de acuerdo con el aprendizaje obtenido del incremento de software funcionando, el cual constituye la única medida objetiva de progreso del proyecto.

Tema 2: La Esencia de las Historias de Usuario (User Stories)

Origen e Definición de las Historias de Usuario

Las **Historias de Usuario** (User Stories) fueron introducidas formalmente por Kent Beck en 1999 dentro del framework Extreme Programming (XP).

En lugar de ser especificaciones de requerimientos rígidas o contratos detallados, las Historias de Usuario se definen como:

"Una expresión de intención; una promesa para mantener una conversación sobre una necesidad del usuario."

Fred Brooks señaló de manera célebre en "No Silver Bullet" que *"la parte más difícil de construir un sistema de software es decidir precisamente qué construir... Ninguna otra parte del trabajo afecta tanto al sistema resultante si se hace incorrectamente y ninguna otra es tan difícil de rectificar más adelante"*. Las historias de usuario nacen para mitigar esta dificultad, enfocándose en la comunicación interpersonal y el entendimiento compartido en lugar de la documentación exhaustiva en prosa que suele ocultar ambigüedades.

La Naturaleza Multipropósito de las Historias de Usuario

Las Historias de Usuario son artefactos versátiles y ligeros que cumplen múltiples funciones simultáneas dentro del ciclo de desarrollo ágil:

Representan una necesidad del usuario: Expresan un problema u oportunidad real desde la perspectiva de quien interactúa con el producto.

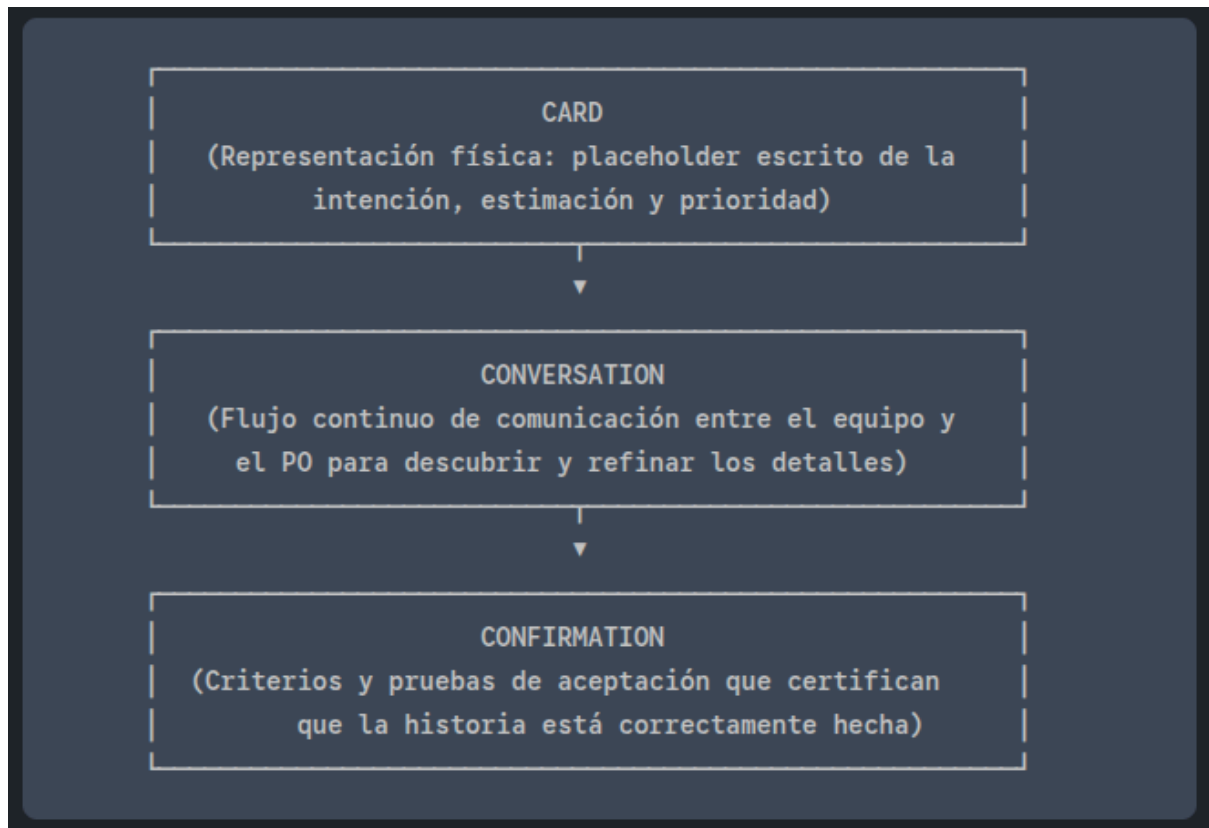
Actúan como una descripción del producto: El conjunto de historias define de manera incremental la funcionalidad del sistema sin requerir extensas especificaciones de diseño previas.

Constituyen un ítem de planificación: Son unidades estimables y acotadas que facilitan la planificación de releases y sprints bajo criterios de capacidad y velocidad.

Funcionan como un Token (ficha) para una conversación: Sirven como un recordatorio físico o digital para que los desarrolladores y el Product Owner se reúnan a discutir los detalles finos de la implementación justo antes de que comience el trabajo.

Actúan como un mecanismo para diferir conversaciones: Permiten postergar el análisis detallado de una característica de baja prioridad simplemente anotándola en la pila de producto como una tarjeta de alto nivel (épica), liberando al equipo de un análisis prematuro.

El Modelo de las 3C de Ron Jeffries



Propuesto originalmente en 2001, este modelo conceptualiza de manera perfecta las tres dimensiones inseparables que conforman una Historia de Usuario para garantizar el entendimiento y la verificación de requerimientos:

Tarjeta (Card): Constituye el elemento visible y tangible de la historia (tradicionalmente manuscrita en tarjetas de índice de papel para facilitar su manipulación colaborativa). La tarjeta no contiene toda la especificación, sino una breve descripción en lenguaje de negocio que sirve como recordatorio y placeholder para planificar y estimar el requerimiento.

Conversación (Conversation): Es la parte más importante y a menudo invisible de la historia. Consiste en el diálogo interactivo, continuo e iterativo entre los desarrolladores, testers y el Product Owner para desentrañar las reglas de negocio y los detalles semánticos del requerimiento. Es a través de la conversación donde se genera el conocimiento tácito y compartido, mitigando el riesgo de malinterpretación de los textos escritos.

Confirmación (Confirmation): Representa el acuerdo final sobre cómo se certifica que la historia ha sido implementada correctamente. Se materializa a través de los **Criterios de Aceptación y las Pruebas de Aceptación**, las cuales definen las expectativas del Product Owner y dictan cuándo se considera que el incremento funcional provee valor real.

Estructura y Formato Estándar de Expresión

Para estandarizar el modelado de requerimientos ágiles y reducir la variación semántica, las Historias de Usuario se redactan siguiendo una plantilla estructurada en tres cláusulas funcionales:

*“**Como** [Rol / Tipo de Usuario] **quiero** [Actividad / Acción], **de forma tal que** [Valor del Negocio / Beneficio]”*

Como [Rol / Tipo de Usuario]: Representa quién realiza la acción o quién recibe el beneficio directo de la funcionalidad. Es crucial utilizar roles específicos de usuario identificados en el dominio del negocio en lugar del genérico "Usuario".

quiero [Actividad / Acción]: Representa qué acción o comportamiento debe realizar el sistema [9]. Describe la interacción lógica en lenguaje del cliente, libre de tecnicismos.

de forma tal que [Valor del Negocio / Beneficio]: Comunica el porqué es necesaria la actividad, justificando el retorno de inversión y proveyendo al equipo técnico el contexto del problema para diseñar la mejor solución técnica.

Ejemplo de Tarjeta de Historia de Usuario:

Título: Buscar Destino por Dirección

Declaración: Como conductor, quiero buscar un destino a partir de una calle y altura para poder llegar al lugar deseado sin perderme.

Modelado de Roles de Usuario y la Tarjeta de Rol

Frecuentemente se comete el error de redactar todas las historias de usuario desde la perspectiva de un único usuario genérico, ignorando la diversidad de participantes y sus necesidades específicas. El agilismo propone el **Modelado de Roles de Usuario** como técnica de análisis previa para identificar sistemáticamente a los actores del sistema.

Se utiliza la **Tarjeta de Rol de Usuario** para caracterizar formalmente a cada participante, describiendo sus habilidades, frecuencia de uso del sistema, contexto operativo y metas personales frente al producto.

Ejemplo de Tarjeta de Rol de Usuario:

Rol de Usuario: Reclutador Interno

Caracterización: No es un experto en computadoras, pero es bastante adepto al uso de la Web. Utilizará el software con poca frecuencia pero con extrema intensidad. Su principal objetivo es leer anuncios de otras compañías para identificar palabras clave efectivas. La facilidad de uso y la recordabilidad a lo largo de los meses son sus atributos de calidad críticos.

Tema 3: Atributos de Calidad y Criterios de Aceptación

El Modelo INVEST (Bill Wake)

Para garantizar que una **Historia de Usuario** esté madura y lista para ser incorporada en una iteración de desarrollo, debe cumplir con los seis atributos de calidad descritos por el acrónimo **INVEST** (Bill Wake, 2003):

I - Independent (Independiente): Las historias deben ser, en la medida de lo posible, autónomas y no presentar dependencias de orden de desarrollo acopladas con otras historias, permitiendo que el Product Owner las planifique e implemente en cualquier orden de acuerdo con las prioridades del negocio.

N - Negotiable (Negociable): Una historia no es un contrato cerrado ni una especificación inalterable. El "qué" se describe en la tarjeta, pero el "cómo" detallado debe ser negociable y surgir del consenso mutuo entre los desarrolladores y el Product Owner durante la conversación.

V - Valuable (Valiosa): Cada historia de usuario debe proveer un incremento de valor de negocio explícito y mensurable para el cliente o el usuario final del sistema. Las historias técnicas de infraestructura que no generan valor de negocio directo deben ser evitadas o reformuladas en términos de su impacto final.

E - Estimatable (Estimable): El equipo de desarrollo debe comprender el alcance y la complejidad de la historia lo suficiente como para poder asignarle un tamaño relativo en puntos de historia. La falta de estimabilidad suele indicar una carencia de conocimiento del dominio de negocio, falta de pericia técnica o un tamaño excesivamente grande del requerimiento.

S - Small (Pequeña): Las historias deben ser de un tamaño lo suficientemente reducido como para ser diseñadas, implementadas, probadas y terminadas por completo dentro de una única iteración o Sprint. Historias muy grandes (épicas) introducen alta incertidumbre y riesgo de arrastre.

T - Testable (Testable): Debe existir una forma objetiva de demostrar empíricamente que la funcionalidad ha sido implementada correctamente mediante criterios y pruebas claras de validación. Si no se puede probar, la historia no es clara y no puede ser aceptada.

Criterios de Aceptación

Los **Criterios de Aceptación** constituyen las condiciones específicas de frontera e hipótesis de negocio acordadas que debe cumplir una Historia de Usuario para que sea catalogada como aceptada por el Product Owner.

Propósito Fundamental:

- Definen con claridad los **límites funcionales y operativos** de una historia de usuario.
- Ayudan al Product Owner a explicitar el alcance mínimo requerido para que la funcionalidad provea el valor de negocio deseado.
- Alinean al equipo técnico bajo una visión compartida de la meta de la historia.
- Sirven como la base directa para que desarrolladores y testers deriven y diseñen los casos de prueba dinámicos.
- Establecen un límite claro que le indica a los desarrolladores exactamente cuándo detener la adición de funcionalidad a una historia.

Atributos de los Criterios de Aceptación de Alta Calidad:

Definen una intención, no una solución: Deben describir qué comportamiento se espera del sistema, no el diseño técnico detallado de la base de datos o la interfaz (ej. "El usuario debe elegir al menos una cuenta para operar" es correcto; "Se debe validar mediante un trigger en la tabla 'cuentas_usuario'" es incorrecto).

Independientes de la implementación: Deben mantenerse estables aun si se cambia la tecnología de soporte o los frameworks del sistema.

Relativamente de alto nivel: No es necesario detallar de forma exhaustiva cada etiqueta gráfica o formato menor en los criterios de aceptación.

Gestión de Detalles Finos e Información Complementaria

Los estudiantes de Ingeniería en Sistemas suelen preguntarse: ¿Dónde registramos los detalles específicos del requerimiento, tales como nombres de columnas, formatos de datos, reglas tipográficas o selecciones de controles de interfaz?.

El enfoque ágil establece que estos detalles no se plasman en la tarjeta de la historia para evitar su sobrecarga y mantenimiento obsoleto. En su lugar, el equipo captura y almacena estos detalles dinámicamente en dos entornos:

Documentación interna y colaborativa del equipo: Wikis, repositorios o herramientas de gestión ágil asociadas a la historia.

Pruebas de aceptación automatizadas: Códigos y especificaciones ejecutables que actúan como la documentación viva, precisa y autoverificable del sistema.

Pruebas de Aceptación y el Enfoque ATDD/STDD

Las Pruebas de Aceptación complementan y cierran la dimensión de la Confirmación de las historias de usuario. Representan los escenarios reales de uso con datos concretos que se ejecutan sobre el incremento funcional para certificar su conformidad.

El proceso de aplicación metodológica consta de dos pasos fundamentales:

Identificación de las pruebas: Se describen los escenarios conceptuales de prueba (normalmente anotados de forma resumida al dorso de la tarjeta física de la historia de usuario) en conjunto con el Product Owner.

Diseño y especificación completa de los casos de prueba: Se formulan las precondiciones, secuencias de acciones, datos de entrada de prueba y resultados esperados para cada escenario identificado.

Ejemplo de Criterios y Pruebas de Aceptación en Tarjeta:

Frente de la Tarjeta (Card):

Como Conductor quiero buscar un destino a partir de una calle y altura para poder llegar al lugar deseado sin perderme.

Criterios de Aceptación:

La altura de la calle es un número entero positivo.

La búsqueda no puede demorar más de 30 segundos en retornar resultados.

Dorso de la Tarjeta (Confirmation):

Prueba de Usuario 1 (Caso Feliz): Probar buscando un destino en un país y ciudad existentes, con calle existente y altura válida. Resultado: El sistema geolocaliza el mapa exitosamente en < 30 segundos.

Prueba de Usuario 2 (Error de Altura): Probar buscando destino con altura ingresada como texto o caracteres especiales. Resultado: El sistema bloquea la acción y emite un mensaje descriptivo de entrada inválida.

Este enfoque se expande hacia el **Desarrollo Dirigido por Pruebas de Aceptación** (ATDD / STDD), técnica ágil de diseño en la cual las especificaciones de los requerimientos se traducen en ejemplos ejecutables formulados en frameworks (como Fitnesse, Concordion o Cucumber) antes de iniciar la codificación del sistema. ATDD transforma el rol de análisis, reemplazando la prosa ambigua por requerimientos vivos que pueden ser validados automáticamente por una máquina en un pipeline de integración continua, garantizando que el equipo técnico construya exactamente lo demandado por el negocio.

Antipatrones de Redacción: Qué "Huele Mal" (Story Smells)

Un estudiante universitario debe ser capaz de diagnosticar de manera técnica cuándo una Historia de Usuario está mal redactada o desvirtúa la filosofía ágil. Mike Cohn y Henrik Kniberg identifican antipatrones críticos denominados "Story Smells":

Huele a restricción técnica o de arquitectura:

Ejemplo: "El software deberá estar desarrollado en JAVA" o "Todos los gráficos se harán con una librería de terceros".

Problema: No expresan un objetivo de negocio ni proveen valor de usuario directo, sino decisiones tecnológicas de implementación que acoplan el diseño técnico de forma prematura. Deben ser tratadas como restricciones no funcionales globales del proyecto, no como historias de usuario aisladas.

Huele a requisito no testeable u objetivo vago:

Ejemplo: "El usuario puede ejecutar el sistema en Windows XP y Linux" o "El usuario puede deshacer hasta 50 comandos".

Problema: Si bien son requerimientos de software válidos, forzarlos rígidamente dentro de la plantilla "Como... quiero... para..." resulta artificial y dificulta la negociación del alcance y su validación independiente.

Huele a restricción contractual encubierta:

Ejemplo: "El software deberá entregarse el 30 de Julio".

Problema: Los hitos del cronograma son variables de la gestión de proyectos (ámbito de la triple restricción), no unidades de funcionalidad incremental del producto.

Tema 4: Estimación de Tamaño y Esfuerzo en Entornos Ágiles

En la Ingeniería de Software tradicional, las estimaciones intentan determinar directamente el **esfuerzo** (horas-persona o días-persona) y el calendario desde el primer día del proyecto. El agilismo separa estrictamente el Tamaño relativo de la funcionalidad del Esfuerzo real requerido para implementarla.

Tamaño vs. Esfuerzo

Tamaño Relativo: Es un atributo intrínseco de la historia de usuario basado en su complejidad técnica, riesgo latente e incertidumbre y el volumen de trabajo involucrado, independientemente de quién sea el desarrollador que la programe. Es una medida abstracta y comparativa (relativa) de volumen.

Esfuerzo: Es el tiempo real de calendario o dedicación requerida (horas, días de trabajo) para codificar e integrar la funcionalidad. El esfuerzo es variable y depende directamente de las habilidades, experiencia, herramientas y familiaridad del desarrollador asignado con el dominio técnico de la solución.

El Concepto de Story Point (Punto de Historia)

El **Story Point** (Punto de Historia) es la unidad métrica adoptada por los equipos ágiles para cuantificar el **tamaño relativo de una historia de usuario**. Es el equivalente a lo que representa "el kilo" en un sistema físico de medición de peso: da una idea del "peso relativo" de cada requerimiento por comparación cruzada.

Escalas de Medición Comunes: Una vez adoptada una escala de estimación, el equipo debe mantenerla de manera inalterable para no alterar la métrica patrón del proyecto. Las escalas más populares son:

- **Tamaño por Números Simples:** Escala lineal de 1 a 10.
- **Talles de Remeras:** Clasificación categórica: S, M, L, XL, XXL.
- **Serie Geométrica 2^n :** Escala exponencial: 1, 2, 4, 8, 16, 32, 64.
- **Sucesión de Fibonacci Modificada:** 0, 1/2, 1, 2, 3, 5, 8, 13, 20, 40, 100, ?. La justificación matemática de usar Fibonacci radica en que **la complejidad de un requerimiento tiende a incrementarse exponencialmente** a medida que este crece en volumen e incertidumbre, impidiendo realizar estimaciones lineales precisas.

Significado Práctico de los Valores Extremos en la Sucesión:

0: No se comprende la funcionalidad o es un cambio cosmético trivial de esfuerzo despreciable.

20: Indica que la historia posee una alta incertidumbre y una complejidad que requiere que el equipo discuta en detalle si no es viable dividirla en unidades menores.

40 / 100: Confirmación absoluta de que la historia es de un tamaño excesivamente grande (épica) que hace inviable su incorporación a un único Sprint. Mejor ni arrancar hasta dividirla.

? (Signo de Interrogación): El estimador carece por completo de la información conceptual del dominio técnico o de negocio como para emitir una estimación.

La Velocidad del Equipo

La Velocidad es la cantidad de puntos de historia relativos que un equipo de desarrollo es capaz de llevar exitosamente al estado "Hecho" (Definition of Done) al finalizar una iteración o Sprint.

La velocidad constituye una métrica histórica y empírica de rendimiento. Al multiplicar la velocidad promedio esperada por el número de iteraciones planificadas, el Product Owner puede deducir de manera matemática y empírica cuánto alcance real del Product Backlog cabe en un release de calendario fijo, mitigando las falsas promesas basadas en la triple restricción convencional.

El Algoritmo de Estimación Colaborativa: Planning Poker

El Planning Poker es una técnica gamificada de estimación cooperativa e interactiva en la cual participa todo el equipo de desarrollo ágil para estimar de manera consensuada y libre de influencias subjetivas el tamaño de las historias.

Prerrequisitos de la Sesión:

Product Backlog Preparado: Se dispone de la lista priorizada de historias de usuario a ser estimadas.

Mazos de Cartas: Cada miembro del equipo estimador posee un mazo físico de cartas con los valores de la escala ágil elegida (ej. Fibonacci Modificada).

Presencia de Roles Clave: El Product Owner asiste a la reunión para detallar el alcance de cada historia y responder de forma inmediata a las consultas del equipo, pero no participa de la estimación (las estimaciones pertenecen estrictamente al equipo de desarrollo).

Mecánica de Ejecución del Algoritmo Paso a Paso:

Establecer el Metro Patrón (Base Story): Al inicio, el equipo selecciona una historia de usuario de referencia bien conocida y consensuada (ej. "Story Z"), a la cual se le asigna un

valor base (típicamente 2 o 3 puntos de historia) para servir como el estándar comparativo de tamaño.

Lectura y Exposición: El Product Owner presenta la historia de usuario a ser estimada.

Discusión y Clarificación: Los estimadores analizan la historia con el Product Owner, formulando preguntas para delimitar el alcance real de la tarjeta.

Selección Privada: Cada estimador selecciona en silencio una carta de su mazo que representa el tamaño relativo de la historia en comparación con la base (Z), colocándola boca abajo sobre la mesa para evitar sesgar o influenciar la decisión de sus compañeros (evitando el efecto anclaje).

Exposición Simultánea: Una vez que todos han seleccionado una carta, las exponen boca arriba al mismo tiempo.

Búsqueda del Consenso:

- **Si hay unanimidad o consenso en el valor:** Ese es el tamaño asignado a la historia.
- **Si existe divergencia significativa en los valores:** No se realiza un promedio matemático simple. En su lugar, el facilitador otorga la palabra a los estimadores que emitieron la estimación más alta y la más baja para que expongan sus hipótesis técnicas y supuestos de riesgo. Tras esta discusión aclaratoria, se repite la votación regresando al paso 4.

Iteración: El algoritmo se repite secuencialmente para cada historia priorizada en el Product Backlog.

Tema 5: Gestión y Estructuración Avanzada de Requerimientos

Niveles de Abstracción de Requerimientos

En proyectos reales de sistemas, los requerimientos ingresan en diferentes niveles de madurez y volumen. El agilismo estructura el Product Backlog bajo una jerarquía conceptual de abstracción basada en el tamaño del ítem:

Temas (Themes): Agrupaciones conceptuales o colecciones de historias y épicas relacionadas por una temática o módulo de negocio común.

Épicas (Epics): Historias de usuario de gran tamaño y alcance amplio. Sirven como marcadores de posición útiles para planificar a largo plazo bloques funcionales masivos sin incurrir en desperdicios de especificación prematura. Una épica debe ser descompuesta secuencialmente en múltiples historias de usuario pequeñas e independientes a medida que se acerca al tope de prioridad en la pila de producto.

Historias de Usuario: Unidades funcionales acotadas que satisfacen el modelo INVEST y pueden ser consumidas enteramente en una iteración, proveyendo valor directo al cliente.

Granos de Arena (Sand Grains): Historias sumamente pequeñas que un programador o pareja de desarrollo puede completar en pocas horas o medio día. Abusar de este nivel en el backlog conduce a la microgestión del proyecto.

Estrategia de Descomposición: Porciones Verticales (Vertical Slices)

Una de las mayores disfunciones metodológicas al intentar estructurar historias de usuario consiste en **descomponer los requerimientos** bajo un enfoque de **Porciones**

Horizontales, asociadas a las capas arquitectónicas del sistema (ej. una historia para diseñar la Base de Datos, una historia para la Lógica de Negocio y una historia para construir la interfaz gráfica de usuario - GUI).

El desarrollo ágil proscribe este enfoque debido a que una porción horizontal no provee ningún valor de negocio de forma aislada. El cliente no puede utilizar ni validar un esquema de base de datos sin una interfaz que lo exponga, ni una interfaz vacía sin datos persistidos.

La estrategia correcta de diseño de requerimientos ágiles requiere estructurar las historias como **Porciones Verticales (Vertical Slices)**:

“Una porción vertical corta transversalmente a través de todas las capas de la arquitectura del software (GUI, Lógica de Negocio y Base de Datos), garantizando que el incremento

final sea una funcionalidad completa y ejecutable que provea un valor real y validable para el usuario al final de la iteración."

Esta estrategia de porciones verticales **optimiza el flujo de valor**, permite obtener **retroalimentación temprana** del mercado y fomenta que los desarrolladores trabajen en equipos multidisciplinarios e independientes, minimizando los tiempos de espera y bloqueos entre especialistas de componentes técnicos aislados.

Spikes Técnicos y Funcionales

Toda historia de usuario posee cierto **grado de incertidumbre y riesgo implícito**. Sin embargo, cuando la falta de conocimiento técnico o funcional del dominio impide que el equipo pueda estimar o siquiera comprender una historia crítica del backlog, se introduce un ítem de trabajo especial denominado **Spike**.

Definición de Spike

"Un tipo especial de historia utilizado para adquirir conocimiento, eliminar el riesgo y mitigar la incertidumbre técnica o funcional de un requerimiento del proyecto."

Clasificación de Spikes

Spikes Técnicas: Orientadas a investigar la viabilidad de enfoques de arquitectura, evaluar el rendimiento potencial de una tecnología, realizar pruebas de concepto de integración o tomar decisiones de "hacer o comprar" un componente.

Spikes Funcionales: Orientadas a mitigar la incertidumbre respecto a cómo interactúa el usuario final con el sistema, evaluando el comportamiento esperado mediante prototipos de bajo costo para recabar feedback de los stakeholders antes de la programación real.

Lineamientos Críticos de Uso para los Spikes

La excepción, no la regla: Toda historia posee incertidumbre natural. El equipo debe aprender a tolerar y resolver cierta incertidumbre dentro de la iteración ordinaria. Los spikes deben reservarse exclusivamente para las incógnitas más grandes, críticas e inabordables, usándose como último recurso.

Son estimables, demostrables y aceptables: Un spike debe ser tratado como cualquier ítem en el Sprint backlog. Debe tener una cota de tiempo estricta (timebox), generar un entregable tangible que demuestre el conocimiento adquirido (ej. un prototipo, un informe de performance o un set de pruebas inicial) y ser formalmente validado al final del Sprint.

Implementación separada: El spike debe ejecutarse en una iteración separada y previa a la implementación de las historias de usuario que se beneficiarán del conocimiento obtenido.

Criterios de Entrada y Salida: DoR y DoD

Para estructurar un flujo predecible de requerimientos ágiles libres de cuellos de botella se definen dos políticas explícitas acordadas entre el equipo y los stakeholders:

Definición de Listo (Definition of Ready - DoR)

Es una cualidad emergente del refinamiento continuo del Product Backlog que actúa como el **criterio de entrada de requerimientos** a la reunión de Sprint Planning. Un ítem del backlog se considera "Listo" para ser planificado cuando cumple con las siguientes condiciones mínimas:

- La historia cumple plenamente con los atributos INVEST.
- Existe claridad absoluta en los Criterios de Aceptación y se ha eliminado la ambigüedad conceptual.
- El tamaño relativo de la historia ha sido estimado y es lo suficientemente pequeño como para caber holgadamente en una iteración.
- Se han mapeado, resuelto o mitigado las dependencias técnicas externas críticas.

Definición de Hecho (Definition of Done - DoD)

Es un compromiso formal y un estándar de calidad corporativa que actúa como el **criterio de salida y cierre de requerimientos** al finalizar la iteración. La DoD abarca todos los requisitos no funcionales, de ingeniería y de aseguramiento de calidad obligatorios que debe satisfacer cada incremento funcional de software liberado en el Sprint. Su cumplimiento es innegociable frente a las presiones de calendario.

Una DoD típica incluye criterios tales como: código compilado sin advertencias, pruebas unitarias ejecutadas al 100%, análisis estático de código aprobado, integración continua (CI) exitosa, pruebas de regresión ejecutadas, documentación técnica de soporte actualizada y validación final por parte del área de control de calidad (QA).